

Lab: Small-Step Operational Semantics

Learning Goals

The primary learning goals of this assignment are to build intuition for the following:

- the distinction between a big-step and a small-step operational semantics;
- evaluation order; and
- substitution and program transformation.

Functional Programming Skills Iteration. Introduction to higher-order functions.

Programming Language Ideas Semantics: evaluation order. Small-step operational semantics. Substitution and program transformation.

Instructions

A version of project files for this lab resides in the public [pppl-lab3](#) repository. Please follow separate instructions to get a private clone of this repository for your work.

You will be replacing `???` or `case _ => ???` in the `Lab3.scala` file with solutions to the coding exercises described below.

Your lab will not be graded if it does not compile. You may check compilation with your IDE, `sbt compile`, or with the “sbt compile” GitHub Action provided for you. Comment out any code that does not compile or causes a failing assert. Put in `???` as needed to get something that compiles without error.

You may add additional tests to the `Lab3Spec.scala` file. In the `Lab3Spec.scala`, there is empty test class `Lab3StudentSpec` that you can use to separate your tests from the given tests in the `Lab3Spec` class. You are also likely to edit `Lab3.worksheet.sc` for any scratch work. You can also use `Lab3.worksheet.js` to write and experiment in a JavaScript file that you can then parse into a JavaScripty AST (see `Lab3.worksheet.sc`).

If you like, you may use this notebook for experimentation. However, **please make sure your code is in `Lab3.scala`; code in this notebook will not be graded.**

Note that there is a section with concept exercises (Section). Make sure to complete the concept exercises in that section and **turn in this file as part of your submission for the concept exercises**. However, all code and testing exercises from other sections are submitted in `Lab3.scala` or `Lab3Spec.scala`.

Recall that you need to switch kernels between running JavaScript and Scala cells.

Small-Step Interpreter: JavaScripty Functions

We consider the same JavaScripty variant as in the previous exercise on big-step operational semantics (**?@sec-big-step-interpreter-exercise**) where the interesting language feature are first-class functions:

```
trait Expr
case class Fun(xopt: Option[String], y: String, e1: Expr) extends Expr // e ::= x?(y) => e1
case class Call(e1: Expr, e2: Expr) extends Expr // e ::= e1(e2)
```

We consider a `Fun` constructor for representing JavaScripty function literals. This version of `Fun` allows for named functions. When a function expression $x(y) \Rightarrow e'$ has a name, then it is can be recursive. As noted previously about recursive functions (**?@sec-recursive-functions**), variable x is an additional formal parameter, and the function body e' may have free variable uses of x . The variable x gets bound to itself (i.e., the function value for $x(y) \Rightarrow e'$) on a function call.

In the abstract syntax representation, the `xopt: Option[String]` parameter in our `Fun` constructor is `None` if there is no identifier present (cannot be used recursively), or `Some(x: String)` if there is an identifier, `x`, present (can be used recursively).

In this lab, we will do two things. First, we will move to implementing a small-step interpreter with a function `step` that takes an `e: Expr` and returns a one-step reduction of `e`. A small-step interpreter makes explicit the evaluation order of expressions. Second, we will remove environments and instead use a language semantics based on substitution. This change will result in static, lexical scoping without needing closures, thus demonstrating another way to fix dynamical scoping.

These two changes are orthogonal, that is, one could implement a big-step interpreter using substitution (as in **?@sec-big-step-substitution**) or a small-step interpreter using environments. Substitution is a fairly simple way to get lexical scoping, but in practice, it is rarely used because it is not the most efficient implementation.

Static Scoping

Exercise 0.1 (Substitute). Since our implementation requires substitution, we begin by implementing `substitute`, which substitutes value `v` for all *free* occurrences of variable `x` in expression `e`:

```
def substitute(e: Expr, v: Expr, x: String): Expr = ???
```

We advise defining `substitute` by induction on `e`. The cases to be careful about are `ConstDecl` and `Fun` because these are the variable binding constructs (as discussed in the reading on substitution in [?@sec-substitution](#)). In particular, calling `substitute` on expression

```
a; { const a = 4; a }
```

with value 3 for variable `a` should return

```
3; { const a = 4; a }
```

not

```
3; { const a = 4; 3 }
```

This function is a helper for the `step` function, but you might want to implement all of the cases of `step` that do not require `substitute` first.

Iteration

Our `step` performs a single reduction step. We may want to test it by repeatedly calling it with an expression until reducing to value. Thus, from a software engineering standpoint, you might want to evolve the `iterate` function described below together with your implementation of `step`.

This idea of repeatedly performing an action until some condition is satisfied is a loop or iteration. We have seen that we can iterate with a tail-recursive helper function. For example, consider the `sumTo` function that sums the integers from 0 to `n`:

```
def sumTo(n: Int): Int = {
  def loop(acc: Int, i: Int): Int = {
    require(n >= 0)
    if (i > n) acc
    else loop(acc + i, i + 1)
  }
  loop(0, 0)
}
sumTo(100)
```

This pattern of repeating something until a condition is satisfied is exceedingly common (e.g., computing the square root using Newton-Raphson approximation until the error is small enough from a previous assignment).

Because this pattern is so common, we want to get practice refactoring this pattern into a library function. This library function will be a higher-order function because it takes the “something” (i.e., what to do in each loop iteration) as a function parameter.

Exercise 0.2 (Iterate with Error Side-Effects). Implement the generic, higher-order library function `iterateBasic`. The `iterateBasic` function repeatedly calls (i.e., iterates) the callback `stepi` until it returns `None` starting from `acc0: A`. Note that `iterateBasic` is generic in the accumulation type `A`. The `stepi` callback takes the current accumulator of type `A` and the iteration number as an `Int` and indicates continuing by returning `Some(acc)` for some next accumulator value `acc`.

```
def iterateBasic[A](acc0: A)(stepi: (A, Int) => Option[A]): A = {
  def loop(acc: A, i: Int): A = ???
  loop(acc0, 0)
}
```

We can test `iterateBasic` by using it with a client like `sumTo`:

```
def sumTo(n: Int) = {
  iterateBasic(0) { case (acc, i) =>
    require(n >= 0)
    if (i > n) None
    else Some(acc + i)
  }
}
sumTo(100)
```

We see how `sumTo` can use `iterateBasic`.

Exercise 0.3 (Iterate with Error Values). One unfortunate aspect of the above is that `sumTo` “exits `iterateBasic` with an error” by throwing an exception (i.e., with the `require(n >= 0)`). Let us refactor `iterateBasic` to allow for explicit error values using `Either[Err, A]`:

```
def iterate[Err, A](acc0: A)(stepi: (A, Int) => Option[Either[Err, A]]): Either[Err, A] = {
  def loop(acc: A, i: Int): Either[Err, A] = ???
  loop(acc0, 0)
}

def sumTo(n: Int): Either[IllegalArgumentException, Int] = {
  iterate(0) { case (acc, i) =>
    if (n < 0) Some(Left(new IllegalArgumentException("requirement failed")))
    else if (i > n) None
    else Some(Right(acc + i))
  }
}

sumTo(100)
sumTo(-1)
```

The `iterate` is now parametrized by an error type `Err` and returns an `Either[Err, A]`. The `stepi` callback should return `None` if it wants to stop normally, `Some(Left(err))` if it wants to stop with an error, and `Some(Right(acc))` if it wants to continue with an accumulator value `acc`.

We can now see how we can use `iterate` as a library function to iterate your `step` implementation. In particular, this is how `iterate` will be used to iterate `step` while adding some debugging output:

```
def iterateStep(e: Expr) = {
  require(closed(e), s"iterateStep: ${e} not closed")
  if (debug) {
    println("-----")
    println("Evaluating with step ...")
  }
  val v = iterate(e) { (e: Expr, n: Int) =>
    if (debug) { println(s"Step $n: $e") }
    if (isValue(e)) None else Some(step(e))
  }
  if (debug) { println("Value: " + v) }
  v
}
```

Of particular interest is the anonymous function passed to `iterate` that calls your implementation of `step`.

Small-Step Interpreter

In this section, we implement the one-step evaluation judgment form $e \rightarrow r$ that says, “Expression e can take one step of evaluation to a step-result r .”

$$\text{step-results } r ::= \text{typeerror } e \mid e'$$

A step-result r is either a `typeerror` e indicating a dynamic type error in attempting to reduce e or a successful one-step reduction to an expression e' .

We represent a step-result r in Scala using a type `Either[DynamicTypeError, Expr]`:

```
case class DynamicTypeError(e: Expr) {  
  override def toString = s"TypeError: in expression $e"  
}  
type Result = Either[DynamicTypeError, Expr] // r ::= typeerror e | e
```

Note that unlike before, `DynamicTypeError` is not an `Exception`, so it cannot be `thrown`.

The small-step semantics that we should implement are given in the section below (Section). The language we implement is JavaScripty with numbers, booleans, strings, **undefined**, printing, and first-class functions. It is a simpler language than the previous lab because we remove type coercions (except to booleans) and replace most coercion cases with dynamic type error `typeerror` e .

Exercise 0.4 (Step without Dynamic Type Checking). We advise first implementing the cases restricted to judgments of the form $e \rightarrow e'$, that is, implement the `Do` and `SEARCH` rules while ignoring the `TypeError` and `PROPAGATE` rules. Start with implementing a `stepBasic` function with type:

```
def stepBasic(e: Expr): Expr = ???
```

That is, just crash with a `MatchError` exception if your `step` encounters any ill-typed expression e .

The suggested practice here is to read some rules, write a few tests for those rules, and implement the cases for those tests.

Exercise 0.5 (Step with Dynamic Type Checking). Then, copy your code from `stepBasic` to `stepCheck`:

```
def stepCheck(e: Expr): Either[DynamicTypeError, Expr] = ???
```

to then add dynamic type checking. You will likely need to refactor your code to satisfy the new types before implementing the `TypeError` and `PROPAGATE` rules.

Exercise 0.6 (To Boolean). You will need to implement a `toBoolean` function to convert JavaScripty values to booleans, following the `TOBOOLEAN` rules in Section .

```
def toBoolean(e: Expr): Boolean = ???
```

However, you will not need any other type coercion functions here.

Notes

- Note that the tests call the `step` function that is originally defined as:

```
//def step(e: Expr): Either[DynamicTypeError, Expr] = Right(stepBasic(e))  
def step(e: Expr): Either[DynamicTypeError, Expr] = stepCheck(e)
```

You can first test `stepBasic` by uncommenting the first line and commenting out the second line.

- Note that the provided tests are minimal. You will want to add your own tests to cover most language features.

Small-Step Operational Semantics

In this section, we give the small-step operational semantics for JavaScripty with numbers, booleans, strings, **undefined**, printing, and first-class functions. We have type coercions to booleans but otherwise use dynamic type error for other cases.

We write $[v/x]e$ for substituting value v for all free occurrences of the variable x in expression e (i.e., a call to `substitute`).

It is informative to compare the small-step semantics used in this lab and the big-step semantics from last homework.

$\boxed{e \rightarrow e'}$	$\frac{\text{DoNEG} \quad n' = -n_1}{-n_1 \rightarrow n'}$	$\frac{\text{DoARITH} \quad n' = n_1 \text{ bop } n_2 \quad \text{bop} \in \{+, -, *, /\}}{n_1 \text{ bop } n_2 \rightarrow n'}$	$\frac{\text{DoPLUSSTRING} \quad str' = str_1 str_2}{str_1 + str_2 \rightarrow str'}$
$\frac{\text{DoINEQUALITYNUMBER} \quad b' = n_1 \text{ bop } n_2 \quad \text{bop} \in \{<, <=, >, >=\}}{n_1 \text{ bop } n_2 \rightarrow b'}$	$\frac{\text{DoINEQUALITYSTRING} \quad b' = str_1 \text{ bop } str_2 \quad \text{bop} \in \{<, <=, >, >=\}}{str_1 \text{ bop } str_2 \rightarrow b'}$		
$\frac{\text{DoEQUALITY} \quad b' = (v_1 \text{ bop } v_2) \quad \text{bop} \in \{===, !==\}}{v_1 \text{ bop } v_2 \rightarrow b'}$	$\frac{\text{DoNOT} \quad v_1 \rightsquigarrow b_1}{!v_1 \rightarrow \neg b_1}$	$\frac{\text{DoANDTRUE} \quad v_1 \rightsquigarrow \mathbf{true}}{v_1 \ \&\& \ e_2 \rightarrow e_2}$	$\frac{\text{DoANDFALSE} \quad v_1 \rightsquigarrow \mathbf{false}}{v_1 \ \&\& \ e_2 \rightarrow v_1}$
$\frac{\text{DoORTRUE} \quad v_1 \rightsquigarrow \mathbf{true}}{v_1 \ \ e_2 \rightarrow v_1}$	$\frac{\text{DoORFALSE} \quad v_1 \rightsquigarrow \mathbf{false}}{v_1 \ \ e_2 \rightarrow e_2}$	$\frac{\text{DoIFTRUE} \quad v_1 \rightsquigarrow \mathbf{true}}{v_1 \ ? \ e_2 : e_3 \rightarrow e_2}$	$\frac{\text{DoIFFALSE} \quad v_1 \rightsquigarrow \mathbf{false}}{v_1 \ ? \ e_2 : e_3 \rightarrow e_3}$
$\frac{\text{DoSEQ}}{v_1, e_2 \rightarrow e_2}$	$\frac{\text{DoPRINT} \quad v_1 \text{ printed}}{\text{console.log}(v_1) \rightarrow \mathbf{undefined}}$	$\frac{\text{DoCONST}}{\mathbf{const} \ x = v_1; e_2 \rightarrow [v_1/x]e_2}$	
$\frac{\text{DoCALL}}{(x) \Rightarrow e_1)(v_2) \rightarrow [v_2/x]e_1}$	$\frac{\text{DoCALLREC} \quad v_1 = (x_1(x_2) \Rightarrow e_1)}{v_1(v_2) \rightarrow [v_1/x_1][v_2/x_2]e_1}$		

Figure 1: The Do rules for JavaScripty with numbers, booleans, strings, **undefined**, printing, and first-class functions.

$\boxed{e \rightarrow e'}$	$\frac{\text{SEARCHUNARY} \quad e_1 \rightarrow e'_1}{uop e_1 \rightarrow uop e'_1}$	$\frac{\text{SEARCHBINARY1} \quad e_1 \rightarrow e'_1}{e_1 \text{ bop } e_2 \rightarrow e'_1 \text{ bop } e_2}$	$\frac{\text{SEARCHBINARY2} \quad e_2 \rightarrow e'_2}{v_1 \text{ bop } e_2 \rightarrow v_1 \text{ bop } e'_2}$
$\frac{\text{SEARCHIF} \quad e_1 \rightarrow e'_1}{e_1 \ ? \ e_2 : e_3 \rightarrow e'_1 \ ? \ e_2 : e_3}$	$\frac{\text{SEARCHPRINT} \quad e_1 \rightarrow e'_1}{\text{console.log}(e_1) \rightarrow \text{console.log}(e'_1)}$		
$\frac{\text{SEARCHCONST} \quad e_1 \rightarrow e'_1}{\mathbf{const} \ x = e_1; e_2 \rightarrow \mathbf{const} \ x = e'_1; e_2}$	$\frac{\text{SEARCHCALL1} \quad e_1 \rightarrow e'_1}{e_1(e_2) \rightarrow e'_1(e_2)}$	$\frac{\text{SEARCHCALL2} \quad e_2 \rightarrow e'_2}{v_1(e_2) \rightarrow v_1(e'_2)}$	

Figure 2: The SEARCH rules for JavaScripty with numbers, booleans, strings, **undefined**, printing and first-class functions.

$$\begin{array}{c}
\boxed{v \rightsquigarrow b} \\
\text{ToBOOLEANNUMFALSE} \\
\frac{n \in \{0.0, -0.0, \text{NaN}\}}{n \rightsquigarrow \mathbf{false}} \\
\text{ToBOOLEANNUMTRUE} \\
\frac{n \notin \{0.0, -0.0, \text{NaN}\}}{n \rightsquigarrow \mathbf{true}} \\
\text{ToBOOLEANBOOLEAN} \\
\frac{}{b \rightsquigarrow b} \\
\\
\text{ToBOOLEANSTRFALSE} \\
\frac{}{"" \rightsquigarrow \mathbf{false}} \\
\text{ToBOOLEANSTRTRUE} \\
\frac{str \neq ""}{str \rightsquigarrow \mathbf{true}} \\
\text{ToBOOLEANUNDEFINED} \\
\frac{}{\mathbf{undefined} \rightsquigarrow \mathbf{false}} \\
\\
\text{ToBOOLEANFUN} \\
\frac{}{x^?(y) \Rightarrow e \rightsquigarrow \mathbf{true}}
\end{array}$$

Figure 3: The ToBOOLEAN rules for JavaScripty with numbers, booleans, strings, **undefined**, and first-class functions.

Do Rules

Search Rules

Coercing to Boolean

Dynamic Typing Rules

Concept Exercises

Make sure to complete the concept exercises in this section and turn in this file as part of your submission. However, all code and testing exercises from other sections are submitted in `Lab3.scala` or `Lab3Spec.scala`.

Exercise 0.7 (Evaluation Order). Consider the small-step operational semantics shown in Section . What is the evaluation order for $e_1 + e_2$? Explain.

Edit this cell:

???

Exercise 0.8 (Changing Evaluation Order). How do we change the rules to obtain the opposite evaluation order?

Edit this cell:

???

$\boxed{e \rightarrow r}$	$\frac{\text{TYPEERRORNEG} \quad v_1 \neq n_1}{-v_1 \rightarrow \text{typeerror}(-v_1)}$	$\frac{\text{TYPEERRORPLUS1} \quad v_1 \neq n_1 \quad v_1 \neq str_1}{v_1 + v_2 \rightarrow \text{typeerror}(v_1 + v_2)}$
$\frac{\text{TYPEERRORPLUSSTRING2} \quad v_2 \neq str_2}{str_1 + v_2 \rightarrow \text{typeerror}(str_1 + v_2)}$	$\frac{\text{TYPEERRORARITH1} \quad v_1 \neq n_1 \quad bop \in \{-, *, /\}}{v_1 \text{ bop } v_2 \rightarrow \text{typeerror}(v_1 \text{ bop } v_2)}$	
$\frac{\text{TYPEERRORARITH2} \quad v_2 \neq n_2 \quad bop \in \{+, -, *, /\}}{n_1 \text{ bop } v_2 \rightarrow \text{typeerror}(n_1 \text{ bop } v_2)}$	$\frac{\text{TYPEERRORINEQUALITY1} \quad v_1 \neq n_1 \quad v_1 \neq str_1 \quad bop \in \{<, <=, >, >=\}}{v_1 \text{ bop } v_2 \rightarrow \text{typeerror}(v_1 \text{ bop } v_2)}$	
$\frac{\text{TYPEERRORINEQUALITYNUMBER2} \quad v_2 \neq n_2 \quad bop \in \{<, <=, >, >=\}}{n_1 \text{ bop } v_2 \rightarrow \text{typeerror}(n_1 \text{ bop } v_2)}$	$\frac{\text{TYPEERRORINEQUALITYSTRING2} \quad v_2 \neq str_2 \quad bop \in \{<, <=, >, >=\}}{str_1 \text{ bop } v_2 \rightarrow \text{typeerror}(str_1 \text{ bop } v_2)}$	
$\frac{\text{TYPEERRORCALL} \quad v_1 \neq x^?(y) \Rightarrow e_1}{v_1(v_2) \rightarrow \text{typeerror}(v_1(v_2))}$	$\frac{\text{PROPAGATEUNARY} \quad e_1 \rightarrow \text{typeerror } e}{uop e_1 \rightarrow \text{typeerror } e}$	$\frac{\text{PROPAGATEBINARY1} \quad e_1 \rightarrow \text{typeerror } e}{e_1 \text{ bop } e_2 \rightarrow \text{typeerror } e}$
$\frac{\text{PROPAGATEBINARY2} \quad e_2 \rightarrow \text{typeerror } e}{v_1 \text{ bop } e_2 \rightarrow \text{typeerror } e}$	$\frac{\text{PROPAGATEIF} \quad e_1 \rightarrow \text{typeerror } e}{e_1 ? e_2 : e_3 \rightarrow \text{typeerror } e}$	$\frac{\text{PROPAGATEPRINT} \quad e_1 \rightarrow \text{typeerror } e}{\text{console.log}(e_1) \rightarrow \text{typeerror } e}$
$\frac{\text{PROPAGATECONST} \quad e_1 \rightarrow \text{typeerror } e}{\text{const } x = e_1 ; e_2 \rightarrow \text{typeerror } e}$	$\frac{\text{PROPAGATECALL1} \quad e_1 \rightarrow \text{typeerror } e}{e_1(e_2) \rightarrow \text{typeerror } e}$	$\frac{\text{PROPAGATECALL2} \quad e_2 \rightarrow \text{typeerror } e}{v_1(e_2) \rightarrow \text{typeerror } e}$

Figure 4: The TYPEERROR and PROPAGATE rules for JavaScripty with numbers, booleans, strings, **undefined**, printing, and first-class functions.

Exercise 0.9 (Using Short-Circuit Evaluation). Give an example that illustrates the usefulness of short-circuit evaluation. Explain your example.

Edit this cell:

???

Exercise 0.10 (Removing Short-Circuit Evaluation). Consider the small-step operational semantics shown in Section . Does $e_1 \ \&\& \ e_2$ short circuit? Explain. If $e_1 \ \&\& \ e_2$ short circuits, give rules that eliminates short circuiting. If it does not short circuit, give the short-circuiting rules.

Edit this cell:

???

Testing

This section has some space to write some tests in our subset of JavaScript. You might want to work on these tests while you are implementing `step`. As before, you will add your tests to `Lab3StudentSpec`. Your interpreter will run the tests against the expected result you provide. We will write three tests, all of these tests must properly parse.

Exercise 0.11 (Test 1: Higher-Order Function). Write a test case that has a function that takes a function value as an argument (i.e., is a higher order function):

???

Exercise 0.12 (Test 2: Recursion). Write a test case that uses recursion

???

Exercise 0.13 (Test 3: Any Test in this variant of JavaScripty). Write another test

???

Notes

- Remember to add these to `Lab3StudentSpec` in `Lab3Spec.scala`.
- Add the JavaScripty code as a string in `jsyStr` and the expected result in `answer`.

Accelerated Component

For the accelerated component of this lab, we will give rules and implement the behavior that enables us to match JavaScript semantics. In particular, we will give rules and implement type coercions for numbers and strings, and we will update our small-step operational semantics to use them.

Additional Type Coercions

Exercise 0.14. Give the inference rules defining the judgment form for coercing a value to a number $v \rightsquigarrow n$

Edit this cell:

???

Exercise 0.15. Give the inference rules defining the judgment form for coercing a value to a string $v \rightsquigarrow str$

Edit this cell:

???

Exercise 0.16. Implement the `toNumber` and `toStr` coercions.

```
def toNumber(v: Expr): Double = ???  
def toStr(v: Expr): String = ???
```

Notes

- If you recall Lab 2, we implemented these functions. They will be the same here, except we must add the rules for functions.

Updating the Small-Step Operational Semantics

Now that we are allowing type coercions, our operational semantics will change. For example, consider the following Do rule:

$$\frac{\text{DoNEG} \quad v \rightsquigarrow n}{-v \longrightarrow -n}$$

This is a new rule for DoNEG, which is read as if v coerces to n , then $-v$ steps to $-n$. Now that we are allowing non-numbers to be coerced and then negated, we no longer have our TYPEERRORNEG rule.

Note that we only need to update Do rules with coercions and remove TYPEERROR rules. We do not need to update the SEARCH or PROPAGAGE rules.

Exercise 0.17. Explain why we only need to update the Do rules with coercions and remove TYPEERROR rules, and we do not need to update the SEARCH or PROPAGAGE rules.

Edit this cell:

???

One rule of particular interest is DoARITH, which we need to split to account for $e_1 + e_2$ being overloaded for numbers and strings. Given this, we need to rewrite DoARITH so it does not include $+$ (and adds coercions), add the rule DoPLUSNUMBER, and alter DoPLUSSTRING to become two rules.

Exercise 0.18. Give these new Do rules

Edit this cell:

???

Similar to this, our DoINEQUALITYNUMBER rule must be split into two and altered:

Exercise 0.19. Give the two new DoINEQUALITYNUMBER₁ and DoINEQUALITYNUMBER₂ rules:

Edit this cell:

???

Once we have all of these rules defined, we notice that most of our typeerror rules no longer result in type errors. Therefore, most of them should be deleted. In fact, the only non-propagate rule left for type errors is TYPEERRORCALL, since we are still not allowed to call something that is not a function (in JavaScript).

Update Step

Exercise 0.20. Now that we have our type conversion functions implemented and our new rules defined, we are ready to update `step`. Implement `stepCoerce` by first copying your code from `stepCheck` and then update based on your new rules.

```
def stepCoerce(e: Expr): Either[DynamicTypeError, Expr] = ???
```

As before, let the rules guide your implementation.

Notes

- None of your other functions should need to be altered.

Submission

If you are a University of Colorado Boulder student, we use Gradescope for assignment submission. In summary,

- ☐ Create a private GitHub repository by clicking on a GitHub Classroom link from the corresponding Canvas assignment entry.
- ☐ Clone your private GitHub repository to your development environment (using the <> Code button on GitHub to get the repository URL).
- ☐ Work on this lab from your cloned repository. Use Git to save versions on GitHub (e.g., `git add`, `git commit`, `git push` on the command line or via VSCode).
- ☐ Submit to the corresponding Gradescope assignment entry for grading by choosing GitHub as the submission method.

You need to have a GitHub identity and must have your full name in your GitHub profile in case we need to associate you with your submissions.